# SHELFWISE – A Demand-Aware, Tamper-Evident Library Management System in Java

**A PROJECT BASED LEARNING (PBL) REPORT**

***Submitted by***

**SAI CHARAN A - 2104251040844**

**ASHFAQ MUHAMMED A - 2104251040096**

***Submitted in partial fulfilment of the requirements***

***for the***

***Project-Based Learning component of Java Programming***

**BACHELOR OF ENGINEERING**

***in***

**COMPUTER SCIENCE AND ENGINEERING**




**CHENNAI INSTITUTE OF TECHNOLOGY, CHENNAI**




**Affiliated to Anna University, Chennai**

**(Autonomous)**

OCTOBER 2026

## Vision of the Institute:

To be an eminent centre for Academia, Industry and Research by imparting knowledge, relevant practices and inculcating human values to address global challenges through novelty and sustainability.

## Mission of the Institute:

**IM1.** To creates next generation leaders by effective teaching learning methodologies and in still Scientifics Park in them to meet the global challenges.

**IM2.** To transform lives through deployment of emerging technology, novelty and sustainability.

**IM3.** To inculcate human values and ethical principles to cater the societal needs.

**IM4.** To contributes towards the research ecosystem by providing a suitable, effective platform for interaction between industry, academia and R & D establishments.

**IM5.** To nurture incubation centres enabling structured entrepreneurship and start-ups.

# DEPARTMENT OF
# COMPUTER SCIENCE AND ENGINEERING

## Vision of the Department:

To Excel in the emerging areas of Computer Science and Engineering by imparting knowledge, relevant practices and inculcating human values to transform the students as potential resources to contribute innovatively through advanced computing in real time situations.

## Mission of the Department:

**DM1.** To provide strong fundamentals and technical skills for Computer Science applications through effective teaching learning methodologies.

**DM2.** To transform lives of the students by nurturing ethical values, creativity and novelty to become Entrepreneurs and establish start-ups.

**DM3.** To habituate the students to focus on sustainable solutions to improve the quality of life and the welfare of the society.

CHENNAI INSTITUTE OF TECHNOLOGY
(Autonomous)

# BONAFIDE CERTIFICATE

This is to certify that the Project–Based Learning report titled **"SHELFWISE – A Demand-Aware, Tamper-Evident Library Management System in Java"** is a Bonafide record of work carried out by **Sai Charan A [2104251040844]**, **Ashfaq Muhammed A [2104251040096]** of the Department of Computer Science and Engineering, Chennai Institute of Technology, as part of the continuous, mentor–guided Project-Based Learning (PBL) component of the Java Programming course during the academic year [2026–2027] under my supervision.

**SIGNATURE**
Dr. S. PAVITHRA, M.E., Ph.D.,
**Professor and Head,**
Dept. of Computer Science and Engineering
Chennai Institute of Technology,
Chennai – 69.

**SIGNATURE**
Ms. S. RAJA PRIYA, M.E.,
**MENTOR**
**Assistant Professor**
Dept. of Computer Science and Engineering
Chennai Institute of Technology,
Chennai – 69.

Submitted for the final review held on …….

**Internal Examiner** **External Examiner**

# DECLARATION

I/We jointly declare that the PBL report on **"SHELFWISE – A Demand-Aware, Tamper-Evident Library Management System in Java"** is the result of original work done by us and best of our knowledge, similar work has not been submitted to **"ANNA UNIVERSITY, CHENNAI"** for the requirement of Degree of **BACHELOR OF ENGINEERING.** This PBL report is submitted on the partial fulfilment of the requirement of the award of Degree of **COMPUTER SCIENCE AND ENGINEERING.**

SIRAGU

**Signature**

Sai Charan A

Ashfaq Muhammed A

Place: Chennai

Date: 02.10.2026

# ACKNOWLEDGEMENT

We wish to express our sincere gratitude to our honourable Chairman **SHRI. P. SRIRAM** for providing immense facilities at our institution.

We are very proud to render our thanks to our Principal **Dr. A. RAMESH M.E., Ph.D.,** for the facilities and the encouragement given by him toward the progress and completion of our project.

We would like to express special thanks and gratitude to our Dean **Dr. V. SRINIVASA RAO M.E., Ph.D.,** who has been a key source of motivation to us throughout the completion of our course and project work.

We proudly render our immense gratitude to the Head of the Department **Dr. S. PAVITHRA M.E., Ph.D.,** for her effective leadership, encouragement and guidance throughout the project.

We would like to extend our thanks to the Project Co-ordinator **Ms. S. RAJA PRIYA M.E.,** Department of Computer Science and Engineering, for their valuable suggestions throughout this project.

We wish to acknowledge the help received from our class advisors **Ms. S. RAJA PRIYA M.E.,** and **Mr. R. RAHUL M.TECH.,** of the Department of Computer Science and Engineering for their valuable suggestions and support toward the successful completion of the project.

Sai Charan A (2104251040844)

Ashfaq Muhammed A (2104251040096)

# ABSTRACT

Most library management systems treat every book the same way: a fixed loan period, a hold queue that moves only as fast as the slowest borrower, a search box that fails on a single typing mistake, and a transaction history that anyone with database access can quietly edit. SHELFWISE is a Java desktop library management system built to address those four weaknesses and to measure each improvement. It is written in Java 21 with Swing for the interface and JDBC over an SQLite database, structured in presentation, service and data-access layers with an abstract Member hierarchy, a LoanPolicy strategy interface and a custom exception hierarchy. The catalogue is loaded from the public goodbooks-10k dataset; ISBN checksum validation found that 65.9% of its 10,000 ISBNs had lost their leading zeros and repaired them automatically. A demand-adaptive loan policy shortens the loan only when other members are waiting; in a year-long, five-seed simulation of 400 members borrowing in proportion to real Goodreads popularity it cut the mean wait for a held title from 17.0 to 11.9 days (29.9%) while leaving loans of uncontested books unchanged, at the cost of more books returned unfinished. Issuing a book uses a single conditional SQL update, which prevented the double issue of a last copy that a read-then-write implementation produced in 100% of concurrent trials. Every issue, return and hold is written to a SHA-256 hash-chained audit ledger that detected and located all 1,000 simulated tampering attempts. A Damerau–Levenshtein search found the intended book for 75.6% of single-typo queries, against 1.6% for substring matching. All 39 JUnit tests pass, with 74.5% line coverage of the non-interface code.



**Keywords:** Library management system, Java, JDBC, Object-oriented design, Loan policy, Concurrency, Hash chain, Approximate string matching.

# TABLE OF CONTENTS

**CHAPTER 1: INTRODUCTION..... 1**
1.1 Background.....1
1.2 Driving Question.....1
1.3 Objectives.....2
1.4 Scope and Limitations.....2
1.5 Novelty and Contributions.....2
**CHAPTER 2: CONCEPT EXPLORATION.....4**
2.1 Related Approaches.....4
2.2 Summary Table.....5
2.3 What This Told Us.....5
**CHAPTER 3: PROJECT PLANNING AND TEAM ORGANISATION.....6**
3.1 Weekly PBL Progress Log.....6
3.2 Requirements.....6
3.3 Feasibility.....7
**CHAPTER 4: ITERATIVE DESIGN AND DEVELOPMENT.....8**
4.1 System Architecture.....8
4.2 Iteration 1 — Baseline.....8
4.3 Iteration 2 — Refinement.....9
4.4 Final Approach.....9
4.5 Testing and Execution.....10
**CHAPTER 5: IMPLEMENTATION.....12**
5.1 Module Description.....12
5.2 Key Code Snippets.....12
5.3 User Interface / Demo.....13
**CHAPTER 6: RESULTS AND DISCUSSION.....17**
6.1 Evaluation Metrics.....17
6.2 Results Across Iterations.....17
6.3 Discussion.....20
6.4 Limitations.....20
**CHAPTER 7: TEAM REFLECTION AND LEARNING OUTCOMES.....22**
7.1 Individual Reflections.....22
7.2 Team Learning.....22
7.3 Course Outcomes — Evidence Summary.....23
**CHAPTER 8: CONCLUSION AND FUTURE SCOPE.....24**
8.1 Conclusion.....24
8.2 Future Scope.....24
**REFERENCES..... 25**

**APPENDIX..... 26**
A.1 Full Source Code.....26
A.2 Complete Weekly PBL Log.....26
A.3 Self and Peer Assessment.....26
A.4 PBL Poster.....27

# LIST OF FIGURES

Figure 4.1 Layered Architecture of SHELFWISE....8
Figure 4.2 Hash-Chained Audit Ledger....10
Figure 5.1 Circulation Desk: Hold Placed for a Busy Title....14
Figure 5.2 Circulation Desk: Loan Shortened Because Members Are Waiting....14
Figure 5.3 Catalogue: Typo-Tolerant Search....15
Figure 5.4 Demand Board....15
Figure 5.5 Audit Ledger: Chain Verified....16
Figure 5.6 Audit Ledger: Tampering Detected and Located....16
Figure 6.1 Fixed vs Demand-Adaptive Loans (Simulated Year)....18
Figure 6.2 Wait Time Against Cost to Borrowers for Nine Policy Settings....18
Figure 6.3 Double Issue of the Last Copy Under Concurrent Requests....18
Figure 6.4 Search Accuracy With and Without Typing Mistakes....19

# LIST OF TABLES

Table 2.1 Summary of Related Approaches.....5
Table 3.1 Weekly PBL Progress Log.....6
Table 3.2 Hardware and Software Requirements.....7
Table 4.1 Configuration and Test Settings.....11
Table 6.1 Results Across Iterations.....17
Table 6.2 Simulated Year of Circulation, Fixed vs Adaptive.....17
Table 6.3 Tamper Detection by Attack Type.....19
Table 6.4 JUnit Tests and Coverage by Package.....19

# LIST OF ABBREVIATIONS

| Abbreviation | Full Form |
|---|---|
| LMS | Library Management System |
| PBL | Project-Based Learning |
| OOP | Object-Oriented Programming |
| JDBC | Java Database Connectivity |
| DAO | Data Access Object |
| SQL | Structured Query Language |
| GUI | Graphical User Interface |
| JVM | Java Virtual Machine |
| JDK | Java Development Kit |
| ISBN | International Standard Book Number |
| CSV | Comma-Separated Values |
| SHA-256 | Secure Hash Algorithm, 256-bit |
| CAS | Compare-And-Set |
| WAL | Write-Ahead Logging |
| CRUD | Create, Read, Update, Delete |
| ILS | Integrated Library System |

# TEAM ROLES AND RESPONSIBILITIES

Both members jointly owned problem framing, the weekly mentor-review cycle, the literature exploration in Chapter 2 and the final report. The division below reflects primary ownership of the build; both members reviewed and tested the full system before each milestone.

| Team Member | Primary Role | Key Responsibilities |
|---|---|---|
| Sai Charan A | Project Lead — Core Logic, Database & Evaluation | Class design (Member hierarchy, records, exception hierarchy); LibraryService with JDBC transactions; LoanPolicy interface and the demand-adaptive policy; atomic issue and the concurrency benchmark; the SHA-256 audit ledger; the circulation simulator and every experiment in Chapter 6. |
| Ashfaq Muhammed A | Interface, Search & Testing | Swing interface with FlatLaf (circulation desk, catalogue, demand board, audit ledger); Damerau–Levenshtein search; catalogue loader with ISBN validation and repair; the JUnit test suite and coverage run; screenshots. |

# CHAPTER 1

# INTRODUCTION

## 1.1 Background

A college or department library runs on a handful of rules: who may borrow how many books, for how long, what happens when someone else is waiting, and what a late return costs. Paper registers enforce these rules inconsistently, and most software replacements simply copy the register into a database. They keep the same fixed loan period for every book, so a best-seller with five people waiting is lent for exactly as long as a book nobody has asked for in years.

Three further problems are easy to overlook until they cause real harm. When two librarians issue the last copy of a book at the same moment, a system that checks availability and then updates it in two separate steps can lend the same copy twice. Imported catalogue data is often damaged; spreadsheets, for example, silently drop the leading zero of an ISBN. And a fines and loans history kept in an ordinary table can be edited by anyone with database access without leaving a trace.

Java suits this problem well. The domain maps onto classes and records, the rules map onto methods that can be unit-tested, JDBC gives direct control over transactions, and Swing delivers a desktop interface that runs on the library's own machine. SHELFWISE uses all of these, and each core Java concept in the course — inheritance, interfaces, polymorphism, exceptions, collections, concurrency, the java.time API and JDBC — has a concrete job in it.

## 1.2 Driving Question

The question the team set out to answer was: **"Can a Java library management system do more than digitise the register — can it make popular books reach readers sooner, stay correct when two librarians act at once, and prove that its own records have not been altered?"**

This narrowed into a buildable task: model the library's rules as classes; implement them over JDBC with transactional, concurrency-safe operations; replace the fixed loan period with a policy that responds to the hold queue; record every transaction in a tamper-evident ledger; and measure each of these against the conventional approach.

## 1.3 Objectives

- To design the library domain using object-oriented principles: an abstract Member class with Student and Faculty subclasses, immutable records for books and loans, a LoanPolicy interface, and a custom exception hierarchy for every rule violation.
- To implement persistent storage with JDBC and PreparedStatement over SQLite, with one transaction per operation and an issue operation that cannot lend the same copy twice under concurrent requests.
- To design a demand-adaptive loan policy and evaluate it against fixed loans in a simulated year of circulation driven by real book-popularity data.
- To record every issue, return and hold in a SHA-256 hash-chained ledger that detects and locates any later alteration.
- To build a Swing interface with typo-tolerant search, validate catalogue ISBNs by checksum, and test the system with a JUnit suite measured by code coverage.
- To document weekly progress and mentor feedback across the PBL cycle and reflect on the team’s division of work.

## 1.4 Scope and Limitations

**In scope.** A single-library desktop application: catalogue of 1,000 titles (1,250 copies), 300 members, issue and return with automatic due dates and fines, a first-come-first-served hold queue, typo-tolerant search, a demand board of the most-wanted titles, and an audit ledger with one-click verification.

**Out of scope.** There is no multi-branch support, web or mobile client, barcode or RFID hardware, e-mail or SMS notification, or online payment; fines are recorded in rupees and settled at the desk. The loan-policy results come from simulation, not a live library, and the ledger detects tampering but cannot prevent someone with full database access from deleting the entire database. These limits are discussed in Section 6.4.

## 1.5 Novelty and Contributions

The project goes beyond a conventional library CRUD application in five measurable ways:

- **Demand-adaptive loans.** The loan period shortens only when members are waiting — in simulation, the mean wait for a held title fell by 29.9% and 5.8% more requests were served, with loans of uncontested books unchanged.

- **Race-free issue.** A single conditional SQL update makes double issue of the last copy impossible; the read-then-write version double-issued in 100% of concurrent trials.
- **Tamper-evident ledger.** Every transaction is SHA-256 hash-chained; all 1,000 simulated edits, deletions and reorderings were detected and located to the exact entry.
- **Data repair on import.** ISBN-10 checksum validation recovered 6,587 of 10,000 ISBNs whose leading zeros had been lost, instead of rejecting two-thirds of the catalogue.
- **Typo-tolerant search.** Damerau–Levenshtein matching found the right book for 75.6% of single-typo queries against 1.6% for substring search, with no loss on clean queries.

# CHAPTER 2

# CONCEPT EXPLORATION

## 2.1 Related Approaches

### *2.1.1 Conventional library systems and loan rules*

Open-source integrated library systems such as Koha define circulation through a matrix of patron category and item type: each cell fixes a loan length, a renewal count and a fine rate. The approach is simple and predictable but blind to demand — a title with a long hold queue keeps the same loan period as one nobody wants. Most student projects follow the same model with a single fixed period.

### *2.1.2 Loan period and book availability*

Operations research on libraries addressed this decades ago. Morse [8] modelled circulation as a queueing system and showed that the probability a wanted book is on the shelf depends on both demand and loan length. Buckland [9] studied book availability at a university library and reported that matching loan periods to demand — shorter loans for heavily requested titles — raised the chance that a reader found the book they wanted. These findings are rarely carried into software, because fixed rules are easier to implement.

### *2.1.3 Correctness under concurrency*

Goetz et al. [3] describe check-then-act as the classic race condition: a thread checks a condition, and before it acts another thread changes it. Issuing the last copy is exactly this pattern. Database systems offer atomic single-statement updates and transactions for this purpose; SQLite documents how each statement commits atomically [13]. Bloch [1] recommends try-with-resources and immutable value classes to keep such code correct.

### *2.1.4 Tamper-evident records*

Haber and Stornetta [4] proposed linking documents by cryptographic hashes so that altering any one breaks every later link — the idea behind today’s blockchains. Schneier and Kelsey [5] applied hash chains to audit logs so that an attacker who gains access cannot modify past entries undetected. A library’s fines and loans history is a natural fit for this technique, which needs no network or third party.

### *2.1.5 Approximate string matching*

Damerau [6] found that over 80% of misspellings are a single insertion, deletion, substitution or transposition of adjacent letters, and Levenshtein [7] defined the edit distance that counts such operations. Matching search words by Damerau–Levenshtein distance therefore tolerates most real typing mistakes.

## 2.2 Summary Table

| Ref. | Approach | Context | Reported Result / Relevance |
|---|---|---|---|
| [8] | Queueing model of circulation | Library operations research | Shelf availability depends on loan length and demand |
| [9] | Demand-matched loan periods | University library study | Shorter loans for popular titles raised availability |
| [3] | Check-then-act analysis | Java concurrency | Separate check and update allows double action |
| [4] | Hash-linked time-stamping | Digital documents | Altering one record breaks all later links |
| [5] | Secure audit logs | Computer forensics | Past log entries cannot be changed undetected |
| [6] | Single-error spelling correction | Misspelling analysis | Over 80% of errors are one edit |

***Table 2.1 — Summary of Related Approaches***

## 2.3 What This Told Us

The exploration gave the project its shape. From [8] and [9] came the central idea: the loan period should respond to the hold queue, and the change should be tested against fixed loans rather than assumed to help. From [3] and [13] came the rule that the availability check and the decrement must be one atomic statement. From [4] and [5] came the audit ledger, chosen because it adds integrity without any server or network. And from [6] came the decision to match search words by Damerau–Levenshtein distance. We stayed with core Java, JDBC and Swing rather than a framework such as Spring and Hibernate, because the course assesses object-oriented design and database connectivity, which a framework would hide.

# CHAPTER 3

# PROJECT PLANNING AND TEAM ORGANISATION

## 3.1 Weekly PBL Progress Log

The project ran across the twelve-week PBL cycle from the Zeroth Review on 27 June 2026, where the team presented the problem statement, to the final testing and report. The table summarises milestones, work completed and the mentor feedback that shaped each iteration.

| Week | Milestone / Task | Work Done | Mentor Remarks |
|---|---|---|---|
| 1–2 | Problem framing (Zeroth Review, 27 June 2026) | Presented the problem statement: a library system for a college library. Listed the library's rules (limits, loan length, holds, fines) and studied existing systems. | Problem approved; asked the team to go beyond basic CRUD and identify a problem real libraries face. |
| 3–4 | Concept exploration, design | Reviewed loan-period research, concurrency and audit-log literature (Chapter 2). Designed the schema, the Member hierarchy and the layered architecture. | Asked for the lending rules to live in one service layer, not in the interface. |
| 5–6 | Iteration 1 — baseline | Implemented the conventional approach: fixed loans, substring search, check-then-act issue, strict ISBN parsing. Loaded the goodbooks-10k catalogue. | Asked why two-thirds of the catalogue was rejected on import; suggested checking the data, not the code. |
| 7–8 | Iteration 2 — refinement | Found the lost leading zeros and added ISBN checksum repair; replaced check-then-act with a conditional SQL update; added transactions, custom exceptions and Damerau–Levenshtein search; started the JUnit suite. | Asked for proof that the issue operation is now safe — a test, not an argument. |
| 9–10 | Final approach | Added the LoanPolicy interface and demand-adaptive policy, the circulation simulator, the SHA-256 audit ledger and the Swing interface. | Asked the team to report the cost of shorter loans honestly, not only the benefit. |
| 11–12 | Testing, evaluation, report | Ran the concurrency, tamper, search and simulation experiments; completed 39 JUnit tests with coverage; captured the interface; compiled this report. | Final system and evaluation accepted for the concluding review. |

***Table 3.1 — Weekly PBL Progress Log***

## 3.2 Requirements

**Data.** The catalogue comes from goodbooks-10k [11], a public dataset of the 10,000 most-rated books on Goodreads with title, authors, year, ISBN-10 and rating count. After ISBN validation and repair, the 1,000 most popular titles were kept, with 3 copies each for the top 50, 2 for the next 150 and 1 for the rest (1,250 copies in all). The rating count drives

simulated demand. The demo database adds 300 members (270 students, 30 faculty) and 60 days of activity generated through the real service layer.

| Category | Requirement |
|---|---|
| Processor / RAM | Any dual-core CPU / 4 GB RAM (developed on an Apple M-series laptop) |
| Programming language | Java 21 (LTS) — records, switch expressions, text blocks |
| Libraries | Java SE (Swing, JDBC, java.time, java.security, java.util.concurrent); sqlite-jdbc 3.46; FlatLaf 3.5 (look and feel); JUnit 5.11; JaCoCo 0.8.12 (coverage) |
| Database | SQLite (embedded, file-based) through JDBC — no server to install |
| Build and run | javac / java with a build script; any IDE (IntelliJ IDEA, VS Code, Eclipse) |
| Version control | Git and GitHub (repository in Appendix A.1) |

***Table 3.2 — Hardware and Software Requirements***

## 3.3 Feasibility

Everything runs on an ordinary laptop with no server: SQLite is embedded, the JDK and libraries are free, and the full experiment suite — five simulated years per policy across eleven policy settings, 2,000 in-memory and 150 database race trials, 1,000 tamper trials and 500 search queries — completes in about fifteen seconds. The work divided cleanly between core logic and interface, with the service layer’s method signatures agreed early so both members could build in parallel.

# CHAPTER 4

# ITERATIVE DESIGN AND DEVELOPMENT

## 4.1 System Architecture

SHELFWISE is organised in layers so that a change in one does not ripple through the others (Figure 4.1). The Swing presentation layer only collects input and shows results. The service layer, LibraryService, holds every rule and runs each operation as one JDBC transaction. It delegates the loan length to a LoanPolicy and writes each event to the audit ledger in the same transaction. The DAO layer performs all SQL through PreparedStatement, and the SQLite database enforces integrity with CHECK constraints and foreign keys.

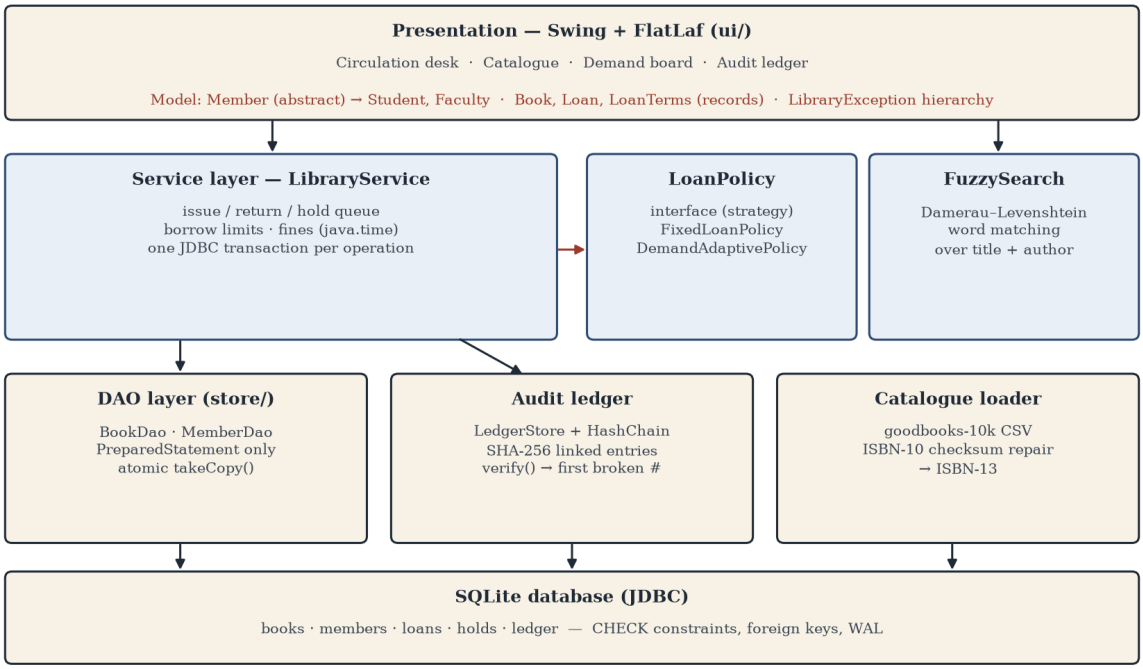



***Figure 4.1 — Layered Architecture of SHELFWISE***

A request flows down: the screen collects a member and a title, the service checks the borrowing limit and the hold queue, the DAO takes a copy atomically, the policy sets the due date, and the ledger records the event. Results and LibraryException messages flow back up, so the librarian sees “Karthik P already has 3 books, the most allowed” rather than an SQL error.

## 4.2 Iteration 1 — Baseline

The first version followed the textbook design that most library projects use, and each part of it is kept in the code as a measurable baseline:

- **Fixed loans** (FixedLoanPolicy): 14 days for students and 28 for faculty, renewable once, regardless of demand.
- **Substring search**: a title or author matches only if it contains the query exactly.
- **Check-then-act issue**: read the available count, and if it is above zero, write back count − 1.
- **Strict ISBN parsing**: an ISBN that is not ten valid characters is rejected.

It worked for the simple path, and the problems appeared as soon as real data and real use were applied. Strict parsing rejected 7,310 of the 10,000 catalogue rows. Substring search found the right book for only 1.6% of queries with one typing mistake. Under concurrent requests, check-then-act issued the last copy more than once in 46.6% of in-memory trials and 100% of database trials. And in simulation, fixed loans left members waiting 17.0 days on average for a held title.

## 4.3 Iteration 2 — Refinement

Iteration 2 fixed the correctness problems. Investigating the rejected rows showed that most ISBNs were nine digits or fewer: the leading zeros had been dropped when the data passed through a spreadsheet. The loader now left-pads a short ISBN and accepts it only if the ISBN-10 checksum then validates, which recovered 6,587 ISBNs (65.9%). As an independent check, 6,571 of the repaired ISBNs (99.8%) match the dataset’s own ISBN-13 column; the 13 that do not point to a different edition in that column. Only 723 rows remained rejected, and every accepted ISBN is converted to ISBN-13.

Issue was rewritten as a single conditional statement — UPDATE books SET available = available − 1 WHERE id = ? AND available > 0 — and the service now treats “one row changed” as success. The availability check and the decrement can no longer be separated. Every operation became a transaction with rollback on any exception; rule violations became typed exceptions (LimitReached, AlreadyHeld, NotOnLoan, InvalidInput, NotFound); and search moved to Damerau–Levenshtein word matching. The JUnit suite was started in this iteration.

## 4.4 Final Approach

**Demand-adaptive loans.** LoanPolicy is an interface with one method, termsFor(member, holdsWaiting, issueDate), returning a LoanTerms record of due date, length, renewability and a plain-language reason. FixedLoanPolicy and DemandAdaptivePolicy are

interchangeable implementations — the Strategy pattern [2] — so the service, the simulator and the tests use either without change. The adaptive rule is:

$$days = max(minDays, round(base / (1 + k \times holdsWaiting)))$$

with base = 14 days for students and 28 for faculty, minDays = 7 and k = 0.5. With nobody waiting the loan is exactly the standard one; with two waiting a student gets 7 days. A shortened loan cannot be renewed, and the reason is shown to the librarian and stored with the loan.

**Hash-chained audit ledger.** Every issue, return and hold appends an entry whose SHA-256 hash covers its sequence number, date, type, details and the previous entry's hash (Figure 4.2). The entry is written inside the same transaction as the change it records, so the two always commit together. Verification recomputes every hash and reports the first entry that fails.

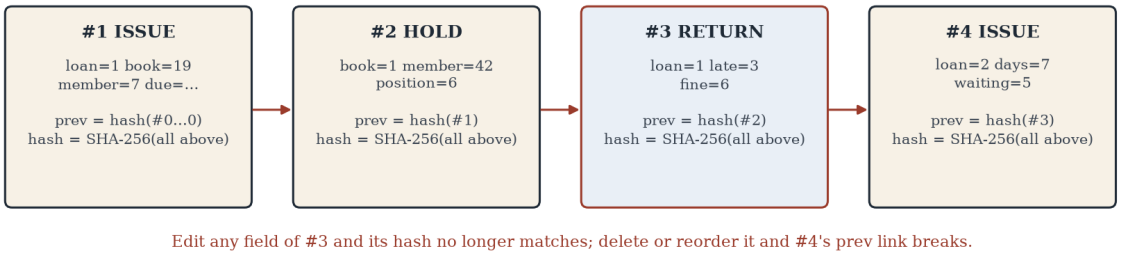



***Figure 4.2 — Hash-Chained Audit Ledger***

**Interface.** A Swing window with a FlatLaf look and feel offers four tabs: a circulation desk with live typo-tolerant search and an explanation of every due date, the catalogue, a demand board of titles with waiting members and their next loan length, and the audit ledger with a Verify button.

## 4.5 Testing and Execution

Testing combined unit tests with four experiments, each designed to measure one claim:

| Component | Configuration |
|---|---|
| JUnit 5 suite | 39 tests in 4 classes: ISBN (8), loan policy and model (10), service with a real SQLite database (13), ledger and search (8); coverage by JaCoCo |
| Circulation simulation | 365 days after a 30-day warm-up; 360 students + 40 faculty; Poisson(45) requests/day; titles chosen in proportion to Goodreads rating count; reading time log-normal (median 10 days, σ 0.6); 25% of overrunning borrowers return late (mean 3 days); renew once if nobody is waiting; seeds 1–5 |
| Policy sensitivity | minDays ∈ {5, 7, 10} × k ∈ {0.25, 0.5, 1.0}, same five seeds |
| Concurrency | 2,000 trials × 16 threads (in-memory); 150 trials × 8 threads on SQLite; one copy, all threads released by a CountDownLatch |

| Component | Configuration |
|---|---|
| Tamper detection | 5,000-entry ledger; 1,000 trials: edit a fine, change a date, delete an entry, swap two entries (250 each) |
| Search | 500 queries of 1–3 consecutive title words, each with one random typing mistake (substitution, deletion, insertion or transposition); seed 11 |

***Table 4.1 — Configuration and Test Settings***

# CHAPTER 5

# IMPLEMENTATION

## 5.1 Module Description

- **model/** — Member (abstract) with Student and Faculty overriding borrowLimit() and baseLoanDays(); Book, Loan and LoanTerms as Java records with validation in compact constructors.
- **policy/** — the LoanPolicy interface, FixedLoanPolicy and DemandAdaptivePolicy.
- **service/** — LibraryService (issue, return, hold queue, fines with java.time.ChronoUnit, queries) and the LibraryException hierarchy.
- **store/** — Database (connection and schema), BookDao, MemberDao, LedgerStore, and Catalogue (CSV loader with ISBN repair).
- **ledger/** — HashChain (SHA-256 hashing and verification) and the LedgerEntry record.
- **search/** — FuzzySearch (Damerau–Levenshtein word matching) and the substring baseline.
- **util/** — Isbn: ISBN-10 and ISBN-13 checksums and conversion.
- **sim/** — CirculationSimulator, a day-by-day discrete-event model of a year of borrowing.
- **ui/** — MainWindow (four Swing tabs) and Screens, which drives the window by its real buttons to capture the figures in this chapter. **app/** — Main, Seeder (demo database) and Experiments (every result in Chapter 6).

## 5.2 Key Code Snippets

Polymorphism in the domain: each member type decides its own limits, and every rule in the system asks the object rather than checking its type.

```
public abstract class Member {
    public abstract int borrowLimit();       // how many books at once
    public abstract int baseLoanDays();      // loan length when nobody is waiting
}
public class Student extends Member {
    @Override public int borrowLimit()  { return 3; }
    @Override public int baseLoanDays() { return 14; }
}
public class Faculty extends Member {
    @Override public int borrowLimit()  { return 6; }
    @Override public int baseLoanDays() { return 28; }
}
```

***Code 5.1 — Member hierarchy (model/)***

The demand-adaptive policy, one implementation of the LoanPolicy interface:

```
@Override
public LoanTerms termsFor(Member m, int holdsWaiting, LocalDate issued) {
    int base = m.baseLoanDays();
    if (holdsWaiting <= 0)                  // nobody waiting: standard loan
        return new LoanTerms(issued.plusDays(base), base, true, "...");
    int days = Math.max(minDays,
            (int) Math.round(base / (1 + k * holdsWaiting)));
    return new LoanTerms(issued.plusDays(days), days, false, "...");
}
```

***Code 5.2 — DemandAdaptivePolicy.termsFor (policy/)***

Taking a copy off the shelf. The check and the decrement are one SQL statement, so two librarians can never issue the last copy twice:

```
public boolean takeCopy(Connection c, int bookId) throws SQLException {
    try (PreparedStatement ps = c.prepareStatement(
            "UPDATE books SET available = available - 1 "
          + "WHERE id=? AND available > 0")) {
        ps.setInt(1, bookId);
        return ps.executeUpdate() == 1;  // 0 rows = no copy was free
    }
}
```

***Code 5.3 — Atomic issue (store/BookDao)***

Verifying the audit ledger:

```
public static int firstBroken(List<LedgerEntry> entries) {
    String prev = GENESIS;                          // 64 zeros
    for (int i = 0; i < entries.size(); i++) {
        LedgerEntry e = entries.get(i);
        String expect = hash(e.seq(), e.timestamp(), e.type(),
                             e.payload(), e.prevHash());
        if (e.seq() != i + 1 || !e.prevHash().equals(prev)
                || !e.hash().equals(expect))
            return i;                               // first broken entry
        prev = e.hash();
    }
    return -1;                                      // chain intact
}
```

***Code 5.4 — Hash-chain verification (ledger/HashChain)***

## 5.3 User Interface / Demo

All screenshots below were produced by driving the real window: the program types into the fields and presses the same buttons a librarian would, then saves the window as an image.

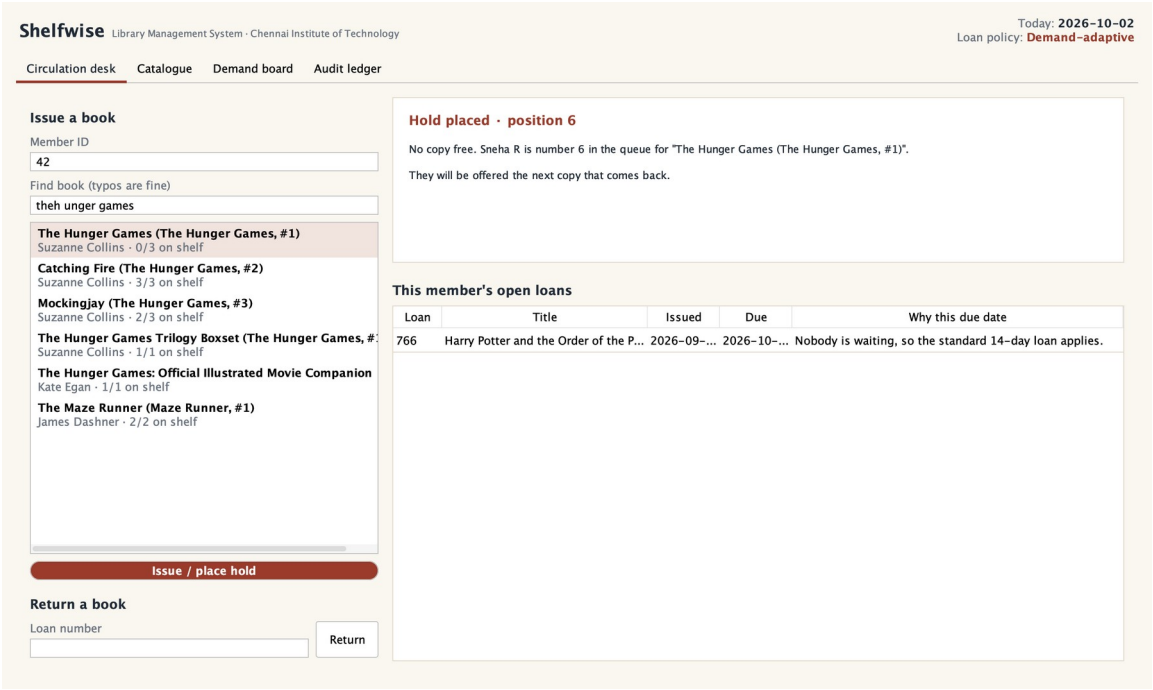



***Figure 5.1 — Circulation Desk: Hold Placed for a Busy Title***

In Figure 5.1 the librarian searches with a typo (“theh unger games”), the correct title is the first match, and because all three copies are out, member 42 is placed sixth in the hold queue.

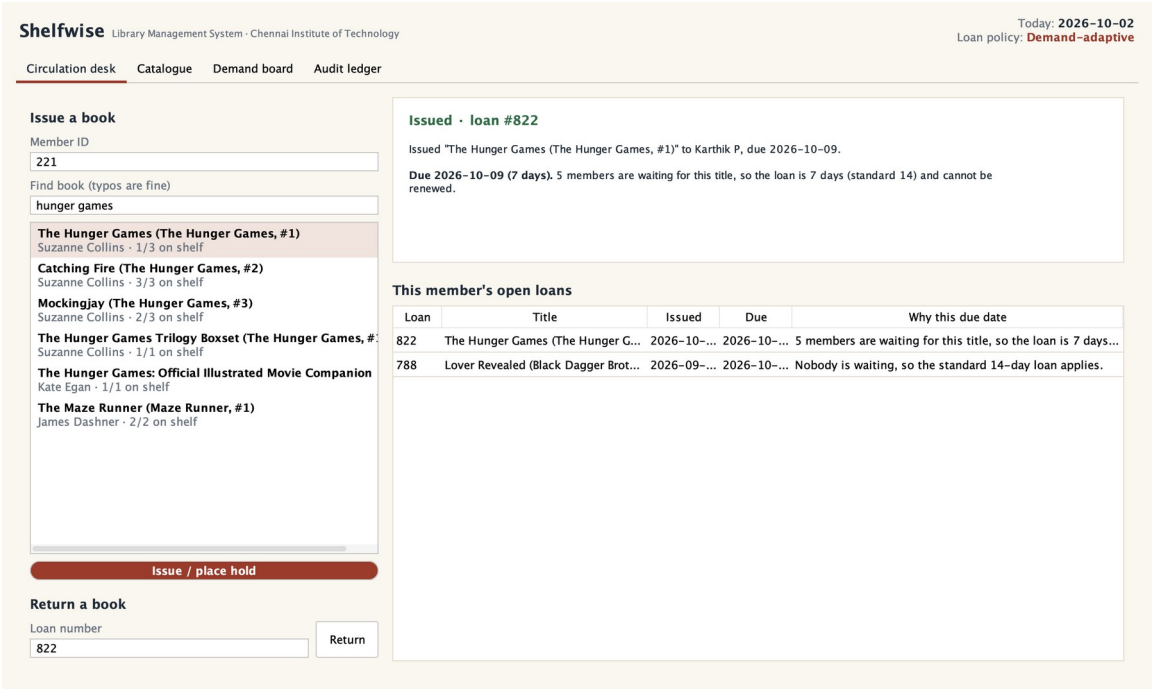
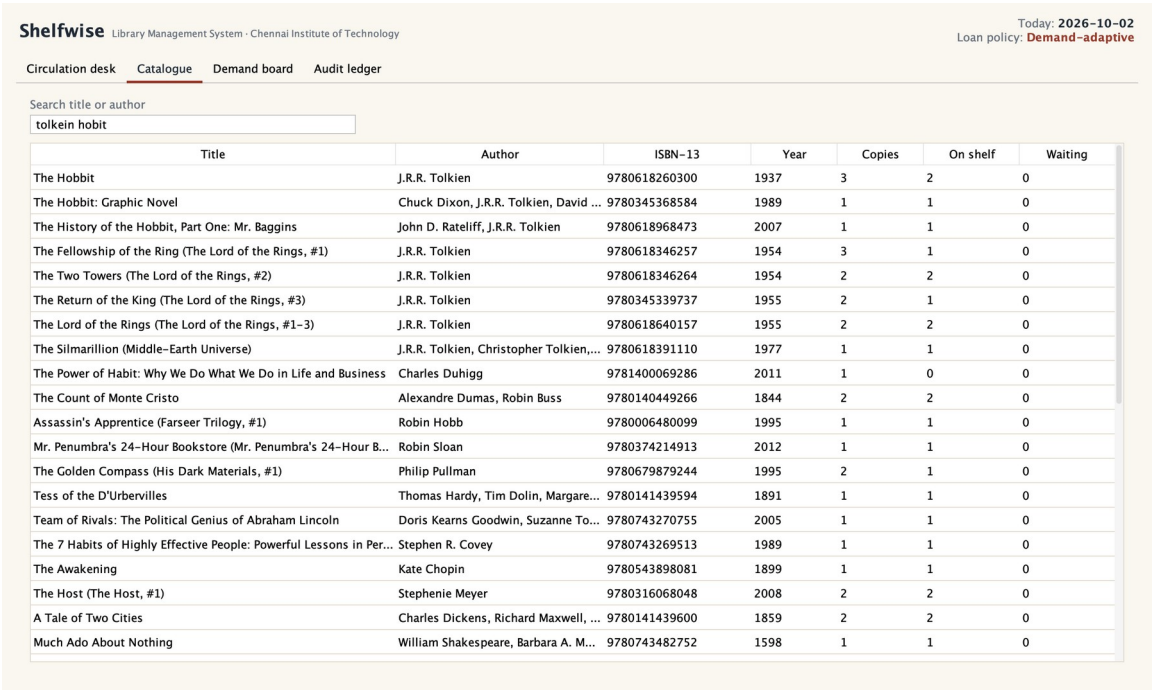



***Figure 5.2 — Circulation Desk: Loan Shortened Because Members Are Waiting***

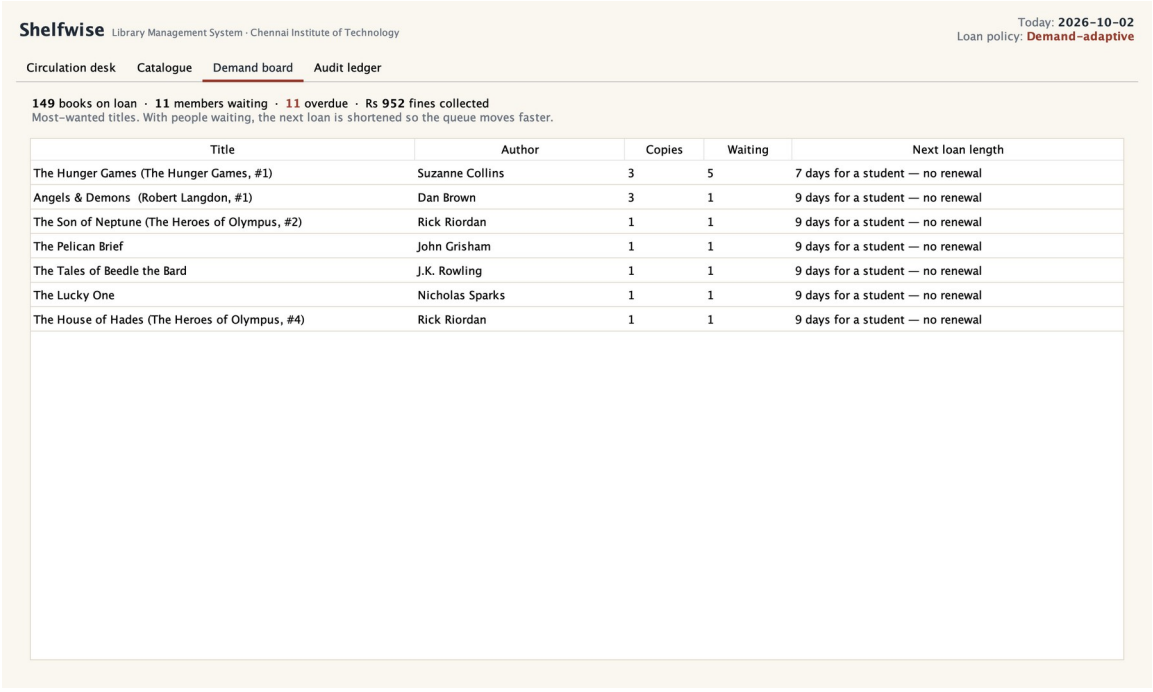
When a copy comes back, the first member in the queue collects it (Figure 5.2). With five members still waiting, the loan is 7 days instead of 14 and cannot be renewed, and the reason

is shown and stored with the loan. The member's other loan, for an uncontested title, kept the standard 14 days.




| Title | Author | ISBN-13 | Year | Copies | On shelf | Waiting |
|---|---|---|---|---|---|---|
| The Hobbit | J.R.R. Tolkien | 9780618260300 | 1937 | 3 | 2 | 0 |
| The Hobbit: Graphic Novel | Chuck Dixon, J.R.R. Tolkien, David ... | 9780345368584 | 1989 | 1 | 1 | 0 |
| The History of the Hobbit, Part One: Mr. Baggins | John D. Rateliff, J.R.R. Tolkien | 9780618968473 | 2007 | 1 | 1 | 0 |
| The Fellowship of the Ring (The Lord of the Rings, #1) | J.R.R. Tolkien | 9780618346257 | 1954 | 3 | 1 | 0 |
| The Two Towers (The Lord of the Rings, #2) | J.R.R. Tolkien | 9780618346264 | 1954 | 2 | 2 | 0 |
| The Return of the King (The Lord of the Rings, #3) | J.R.R. Tolkien | 9780345339737 | 1955 | 2 | 1 | 0 |
| The Lord of the Rings (The Lord of the Rings, #1–3) | J.R.R. Tolkien | 9780618640157 | 1955 | 2 | 2 | 0 |
| The Silmarillion (Middle-Earth Universe) | J.R.R. Tolkien, Christopher Tolkien,... | 9780618391110 | 1977 | 1 | 1 | 0 |
| The Power of Habit: Why We Do What We Do in Life and Business | Charles Duhigg | 9781400069286 | 2011 | 1 | 0 | 0 |
| The Count of Monte Cristo | Alexandre Dumas, Robin Buss | 9780140449266 | 1844 | 2 | 2 | 0 |
| Assassin's Apprentice (Farseer Trilogy, #1) | Robin Hobb | 9780006480099 | 1995 | 1 | 1 | 0 |
| Mr. Penumbra's 24-Hour Bookstore (Mr. Penumbra's 24-Hour B... | Robin Sloan | 9780374214913 | 2012 | 1 | 1 | 0 |
| The Golden Compass (His Dark Materials, #1) | Philip Pullman | 9780679879244 | 1995 | 2 | 1 | 0 |
| Tess of the D'Urbervilles | Thomas Hardy, Tim Dolin, Margare... | 9780141439594 | 1891 | 1 | 1 | 0 |
| Team of Rivals: The Political Genius of Abraham Lincoln | Doris Kearns Goodwin, Suzanne To... | 9780743270755 | 2005 | 1 | 1 | 0 |
| The 7 Habits of Highly Effective People: Powerful Lessons in Per... | Stephen R. Covey | 9780743269513 | 1989 | 1 | 1 | 0 |
| The Awakening | Kate Chopin | 9780543898081 | 1899 | 1 | 1 | 0 |
| The Host (The Host, #1) | Stephenie Meyer | 9780316068048 | 2008 | 2 | 2 | 0 |
| A Tale of Two Cities | Charles Dickens, Richard Maxwell, ... | 9780141439600 | 1859 | 2 | 2 | 0 |
| Much Ado About Nothing | William Shakespeare, Barbara A. M... | 9780743482752 | 1598 | 1 | 1 | 0 |

***Figure 5.3 — Catalogue: Typo-Tolerant Search***




| Title | Author | Copies | Waiting | Next loan length |
|---|---|---|---|---|
| The Hunger Games (The Hunger Games, #1) | Suzanne Collins | 3 | 5 | 7 days for a student — no renewal |
| Angels & Demons (Robert Langdon, #1) | Dan Brown | 3 | 1 | 9 days for a student — no renewal |
| The Son of Neptune (The Heroes of Olympus, #2) | Rick Riordan | 1 | 1 | 9 days for a student — no renewal |
| The Pelican Brief | John Grisham | 1 | 1 | 9 days for a student — no renewal |
| The Tales of Beedle the Bard | J.K. Rowling | 1 | 1 | 9 days for a student — no renewal |
| The Lucky One | Nicholas Sparks | 1 | 1 | 9 days for a student — no renewal |
| The House of Hades (The Heroes of Olympus, #4) | Rick Riordan | 1 | 1 | 9 days for a student — no renewal |

***Figure 5.4 — Demand Board***

The catalogue search "tolkein hobit" (two misspellings) returns The Hobbit first (Figure 5.3). The demand board (Figure 5.4) lists titles with members waiting and the length of the next loan for each.

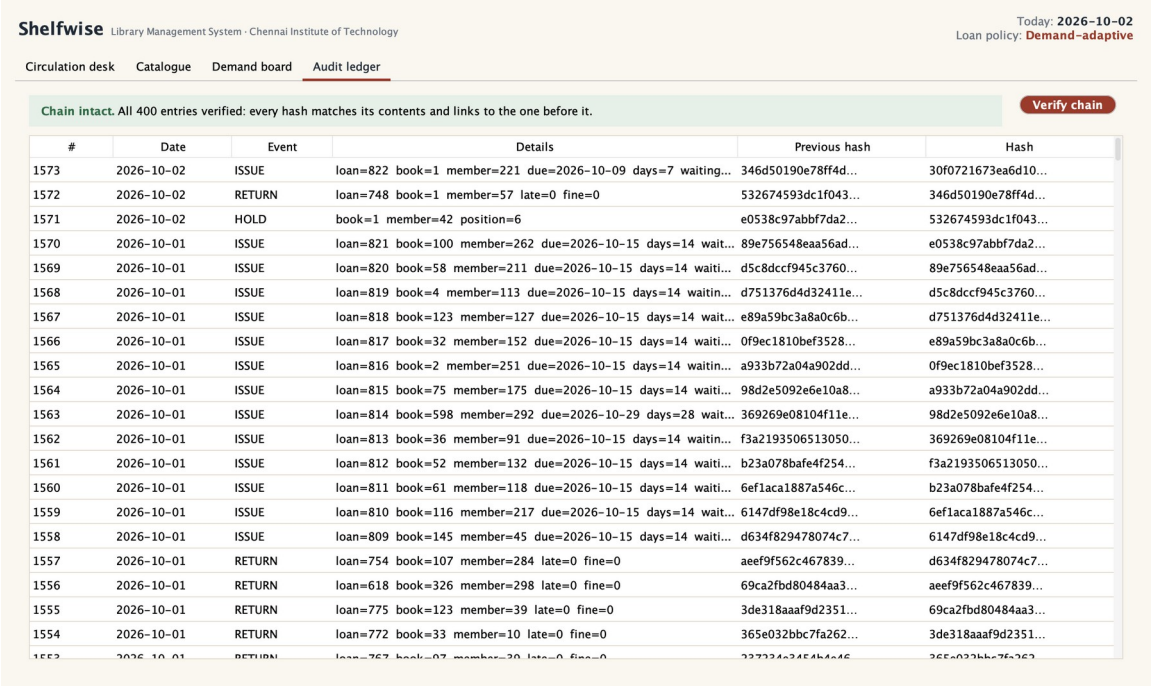



***Figure 5.5 — Audit Ledger: Chain Verified***

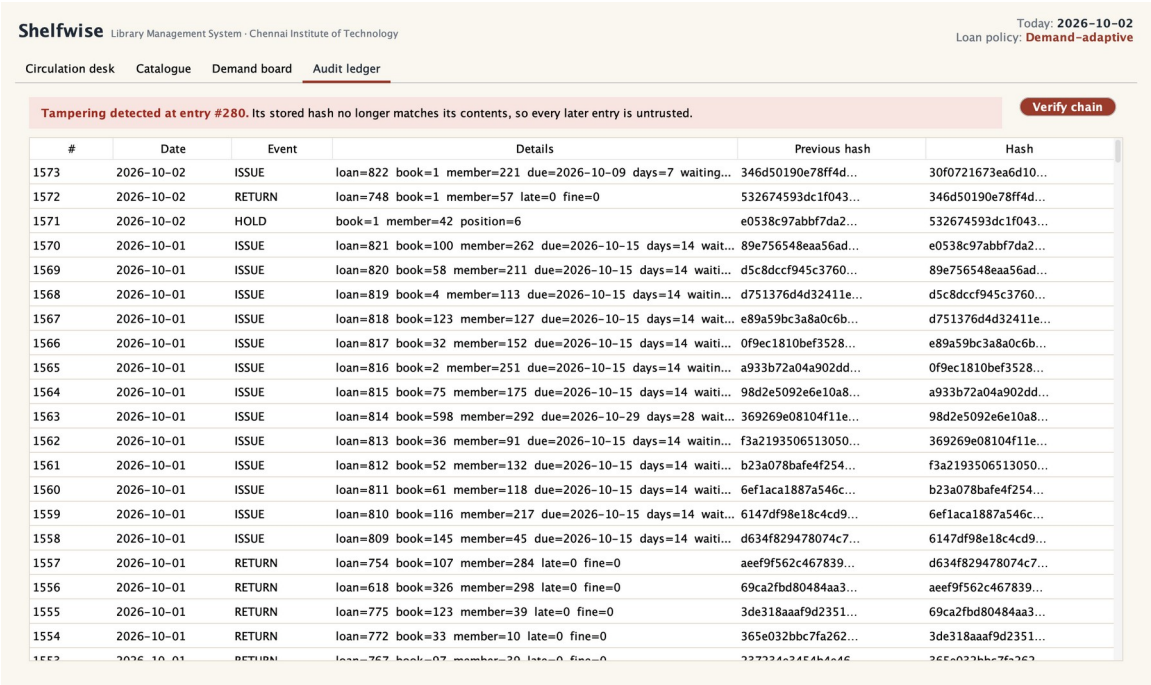



***Figure 5.6 — Audit Ledger: Tampering Detected and Located***

Figure 5.5 shows the verified chain. For Figure 5.6, one recorded fine was altered directly in the database with an SQL UPDATE, bypassing the application entirely — exactly what a dishonest insider could do. Pressing Verify flags the altered entry by number.

# CHAPTER 6

# RESULTS AND DISCUSSION

## 6.1 Evaluation Metrics

Each claim was measured by the metric that matches it. **Test pass rate and line/branch coverage** show that the rules behave as specified and how much code the tests reach. For the loan policy, **mean and 90th-percentile hold wait** measure what readers experience, **requests served** measures throughput, and **loans returned unfinished**, **late returns** and **fines** measure the cost of shorter loans to borrowers — a policy that only reported the benefit would be misleading. For concurrency, the measure is the **share of trials in which one copy was issued more than once**, which must be zero. For the ledger it is the **detection rate and exact location rate** of tampering, and for search the **top-1 and top-5 hit rate** with and without typing mistakes. All experiments are seeded and re-runnable with one command.

## 6.2 Results Across Iterations

| Measure | Iteration 1 (baseline) | Iteration 2 (refinement) | Final approach |
|---|---|---|---|
| Catalogue ISBNs accepted | 2,690 / 10,000 (26.9%) | 9,277 / 10,000 (92.8%) | same |
| Last copy issued twice (JDBC, 8 threads) | 150 / 150 trials | 0 / 150 trials | same |
| Search, one typo, top-1 | 1.6% | 75.6% | same |
| Mean hold wait (simulated) | 17.0 days | 17.0 days | 11.9 days |
| Tampering detected | not recorded | not recorded | 1,000 / 1,000 |
| JUnit tests passing | — | suite started | 39 / 39 |

***Table 6.1 — Results Across Iterations***

Table 6.1 shows each iteration solving a different problem. Iteration 2 fixed correctness: data import, concurrency and search. The final approach changed how the library behaves — how quickly popular books reach readers — and made its history verifiable.

| Measure (mean of 5 seeds) | Fixed loans | Demand-adaptive | Change |
|---|---|---|---|
| Mean wait for a held title | 16.99 days | 11.92 days | −29.9% |
| 90th-percentile wait | 31.6 days | 23.2 days | −26.6% |
| Requests served in the year | 13,232 | 13,993 | +5.8% |
| Loans handed over from the hold queue | 4,414 | 4,876 | +10.5% |
| Mean length of uncontested loans | 15.7 days | 15.6 days | ≈ unchanged |
| Loans returned unfinished | 4.1% | 10.1% | higher (cost) |
| Late returns | 1.1% | 2.5% | higher (cost) |
| Fines in the year | Rs 897 | Rs 2,433 | higher (cost) |

***Table 6.2 — Simulated Year of Circulation, Fixed vs Adaptive***

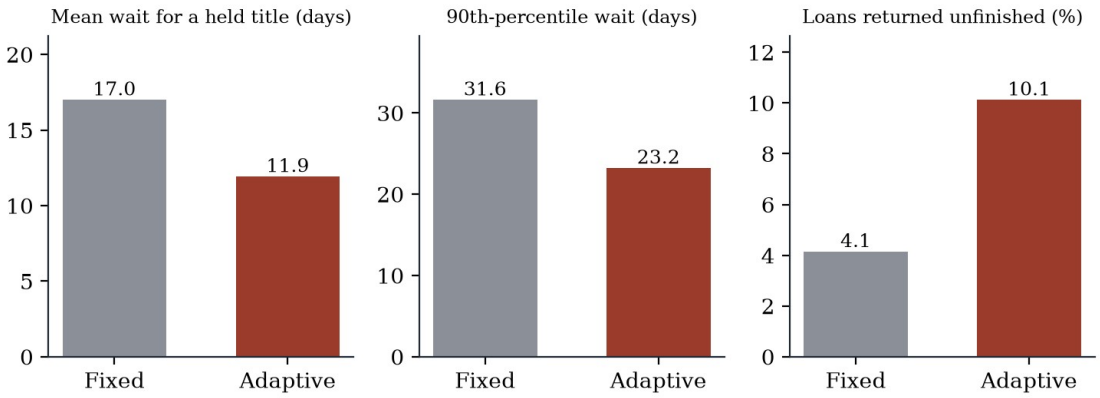



***Figure 6.1 — Fixed vs Demand-Adaptive Loans (Simulated Year)***

Adaptive loans cut the average wait for a held title by 29.9% and the 90th-percentile wait from 32 to 23 days. Because members spend less time blocked in queues, 5.8% more requests were served across the year. Loans of books nobody was waiting for kept their standard length. The cost is real: more borrowers of popular titles had to return a book before finishing it (4.1% → 10.1% of loans), and late returns and fines rose. Figure 6.2 shows how the policy's two parameters trade these against each other.

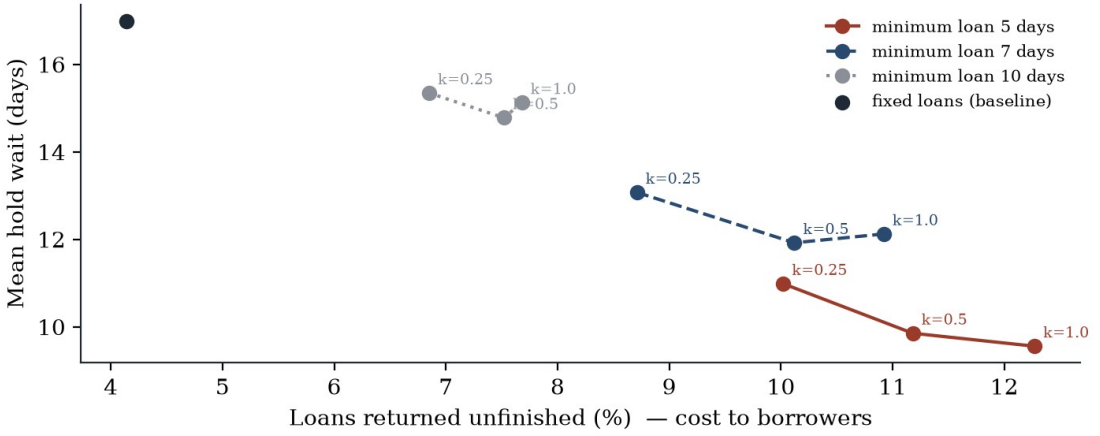



***Figure 6.2 — Wait Time Against Cost to Borrowers for Nine Policy Settings***

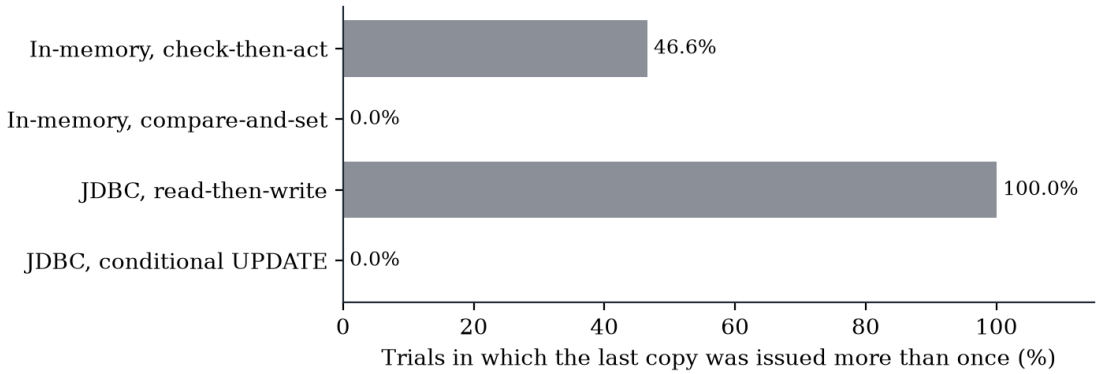



***Figure 6.3 — Double Issue of the Last Copy Under Concurrent Requests***

Figure 6.3: with check-then-act, the last copy was issued more than once in 46.6% of 2,000 in-memory trials (1,030 phantom loans) and in every one of the 150 database trials. The compare-and-set and conditional-UPDATE versions never did.

| Attack on the ledger | Trials | Detected | Located to the exact entry |
|---|---|---|---|
| Edit fine | 250 | 250 | 250 |
| Delete | 250 | 250 | 250 |
| Swap | 250 | 250 | 250 |
| Edit date | 250 | 250 | 250 |

***Table 6.3 — Tamper Detection by Attack Type***

Every one of the 1,000 attacks was detected and located (Table 6.3). Verifying all 5,000 entries took 9.4 ms, so verification can run every time the ledger tab is opened.

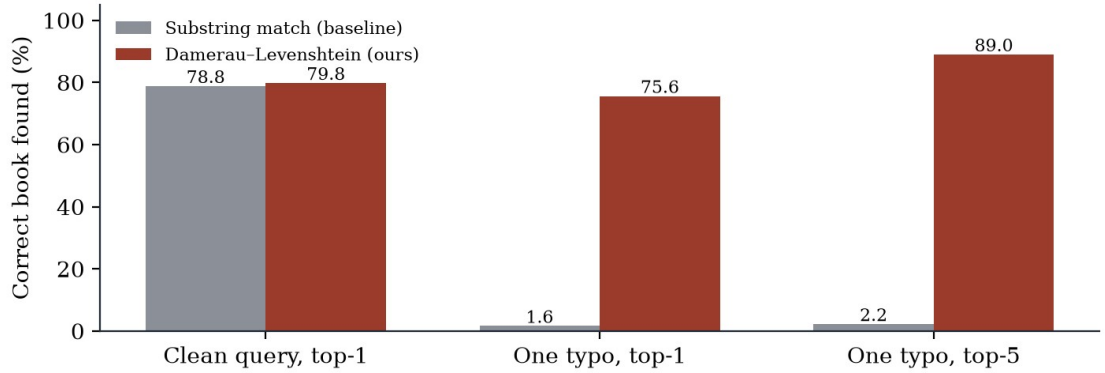



***Figure 6.4 — Search Accuracy With and Without Typing Mistakes***

With one typing mistake, substring search found the intended book first in only 1.6% of 500 queries; Damerau–Levenshtein matching found it first in 75.6% and within the top five in 89.0%, at 5.3 ms per query over 1,000 titles. On clean queries the two are equal (78.8% and 79.8%); the remaining misses are short queries like “The Lord” that genuinely match several books.

| Package | Lines | Line coverage | Branch coverage |
|---|---|---|---|
| util | 21 | 100.0% | 85.7% |
| search | 43 | 100.0% | 84.2% |
| ledger | 15 | 86.7% | 100.0% |
| model | 25 | 84.0% | 55.6% |
| policy | 20 | 80.0% | 87.5% |
| service | 119 | 76.5% | 84.8% |
| store | 121 | 54.5% | 18.0% |
| sim | 105 | 0.0% | 0.0% |
| app | 233 | 0.0% | 0.0% |
| ui | 316 | 0.0% | 0.0% |

***Table 6.4 — JUnit Tests and Coverage by Package***

All 39 tests pass. Coverage of the code that holds the rules — util, search, ledger, model, policy, service and store — is 74.5% of lines and 65.8% of branches. The interface, simulator and application packages are exercised by the screenshot run and the experiments rather than unit tests, which brings overall line coverage to 26.6%.

## 6.3 Discussion

**The policy changes who waits, not how much reading happens.** Shortening loans on contested titles moves time from the current borrower to the people queuing behind them. That is a deliberate fairness choice: a best-seller with five people waiting is a shared resource, while an uncontested book is not, and the policy treats them differently. The sensitivity study makes the choice explicit — a librarian who wants the shortest queues can choose a 5-day minimum and k = 1.0 (a mean wait of 9.6 days), and one who wants to protect readers can choose a 10-day minimum (wait 14.8 days, unfinished loans 7.5%).

**Correctness failures were invisible until they were measured.** The check-then-act version passed every single-user test, and the ISBN problem looked like bad data until the checksum showed the numbers were recoverable. Both were found only by testing under realistic conditions — many threads, the real catalogue — which is why the experiments in this chapter use real data and real concurrency rather than toy examples.

**Integrity is cheap.** The ledger adds one SHA-256 hash per transaction and verifies five thousand entries in about ten milliseconds, yet it turns a silently editable fines table into one where any alteration is detected and pinpointed.

## 6.4 Limitations

- **Simulated circulation.** The loan-policy results come from a model of borrower behaviour (reading times, lateness, renewals), not a live library, and the renewal decision is made when the book is issued. The direction of the effect is robust across nine settings and five seeds; its exact size in a real library is not known.
- **Popularity data.** Demand follows Goodreads rating counts, which reflect a global readership, not a college library’s; a technical library would see a different pattern.
- **The ledger detects but cannot prevent.** Someone with full database access could delete the whole ledger or rebuild it from scratch; anchoring the latest hash outside the database (printed or e-mailed daily) would close that gap.

- **Single machine.** SQLite and Swing suit one library desk; several branches would need a database server and a networked client.
- **Interface not unit-tested.** The Swing code is exercised by the scripted screenshot run but not by assertions, so overall line coverage is 26.6%.

# CHAPTER 7

# TEAM REFLECTION AND LEARNING OUTCOMES

## 7.1 Individual Reflections

**Sai Charan A:** I led the project and built the core: the class design, the service layer with JDBC transactions, the loan policies, the atomic issue, the audit ledger, the simulator and all the experiments. The lesson I value most is that code which passes every test can still be wrong. Our check-then-act issue worked perfectly with one user and failed in every concurrent database trial. Seeing that number changed how I think about shared state. Designing LoanPolicy as an interface was the other turning point: once the policy was a separate object, the same service could run in the app, the simulator and the tests, which is what made the comparison fair.

**Ashfaq Muhammed A:** I built the Swing interface, the typo-tolerant search, the catalogue loader and the JUnit suite. The ISBN problem taught me to question data before code: two-thirds of the catalogue looked invalid until the checksum showed the numbers had only lost their leading zeros. On the interface, the hardest part was making every rule visible — showing why a loan is 7 days instead of 14 — so the librarian can explain it to the member. Writing tests against a real SQLite database rather than mocks also caught problems that simpler tests missed.

## 7.2 Team Learning

Agreeing the service layer’s method signatures early let the interface and the core be built in parallel; the interface could be developed against the real service from the start. Keeping each baseline in the code — the fixed policy, substring search, the naive issue — meant every improvement in Chapter 6 is a measured comparison rather than a claim.

Two mentor remarks changed our direction. The first, at the Iteration 1 review, was to examine the data rather than the code when two-thirds of the catalogue was rejected, which led to the ISBN repair. The second was to report the cost of shorter loans honestly; that is why Table 6.2 lists unfinished loans and fines next to the wait-time gain. If we restarted, we would write the concurrency test before the issue code and the simulator before choosing the policy parameters.

## 7.3 Course Outcomes — Evidence Summary

- **Object-oriented design:** abstract classes and inheritance (Member → Student, Faculty), interfaces and polymorphism (LoanPolicy), encapsulation, immutable records and a custom exception hierarchy (Chapters 4–5).
- **Database connectivity:** JDBC with PreparedStatement throughout, transactions with rollback, the DAO pattern, and constraints enforced in the schema.
- **Concurrency and correctness:** a measured race condition and two correct fixes (compare-and-set and a conditional SQL update), tested with ExecutorService and CountDownLatch.
- **Standard library depth:** java.time for due dates and fines, java.security for SHA-256, java.util.concurrent, collections, records, text blocks and switch expressions.
- **Testing and teamwork:** 39 JUnit tests with coverage, a weekly log of mentor feedback (Table 3.1) and a clear division of work.

# CHAPTER 8

# CONCLUSION AND FUTURE SCOPE

## 8.1 Conclusion

SHELFWISE shows that a Java library management system can do more than digitise the register. Its demand-adaptive loan policy shortened the wait for popular books by 29.9% in a simulated year while leaving uncontested loans unchanged, and the cost to borrowers is measured and adjustable. A single conditional SQL update eliminated a double-issue race that the naive approach hit in 100% of concurrent trials. A SHA-256 hash chain detected and located all 1,000 tampering attempts. ISBN checksum repair recovered 6,587 damaged catalogue records, and typo-tolerant search found the intended book 75.6% of the time instead of 1.6%. All 39 unit tests pass. The driving question is answered: with careful object-oriented design, JDBC transactions and measured experiments, the system makes popular books reach readers sooner, stays correct under concurrent use, and can prove its records are intact.

## 8.2 Future Scope

- Pilot the adaptive policy in the department library and compare real wait times with the simulation.
- Anchor the ledger’s latest hash outside the database each day so that wholesale deletion is also detectable.
- Move to a client–server design (PostgreSQL and a REST API) for several branches and a member-facing web or mobile view.
- Notify the next member in the queue by e-mail or SMS when a held copy is returned.
- Learn each title’s typical reading time from past loans and use it to set fairer adaptive loan lengths.

# REFERENCES

[1] J. Bloch, Effective Java, 3rd ed. Boston, MA, USA: Addison-Wesley, 2018.

[2] E. Gamma, R. Helm, R. Johnson, and J. Vlissides, Design Patterns: Elements of Reusable Object-Oriented Software. Reading, MA, USA: Addison-Wesley, 1994.

[3] B. Goetz, T. Peierls, J. Bloch, J. Bowbeer, D. Holmes, and D. Lea, Java Concurrency in Practice. Boston, MA, USA: Addison-Wesley, 2006.

[4] S. Haber and W. S. Stornetta, “How to time-stamp a digital document,” Journal of Cryptology, vol. 3, no. 2, pp. 99–111, 1991.

[5] B. Schneier and J. Kelsey, “Secure audit logs to support computer forensics,” ACM Trans. Information and System Security, vol. 2, no. 2, pp. 159–176, 1999.

[6] F. J. Damerau, “A technique for computer detection and correction of spelling errors,” Communications of the ACM, vol. 7, no. 3, pp. 171–176, 1964.

[7] V. I. Levenshtein, “Binary codes capable of correcting deletions, insertions, and reversals,” Soviet Physics Doklady, vol. 10, no. 8, pp. 707–710, 1966.

[8] P. M. Morse, Library Effectiveness: A Systems Approach. Cambridge, MA, USA: MIT Press, 1968.

[9] M. K. Buckland, Book Availability and the Library User. New York, NY, USA: Pergamon Press, 1975.

[10] International ISBN Agency, ISBN Users’ Manual, 7th ed. London, U.K., 2017.

[11] Z. Zając, “goodbooks-10k: a new dataset for book recommendations,” GitHub repository, 2017. [Online]. Available: https://github.com/zygmuntz/goodbooks-10k

[12] Oracle, “JDBC Basics,” The Java Tutorials. [Online]. Available: https://docs.oracle.com/javase/tutorial/jdbc/basics/

[13] SQLite Consortium, “Atomic Commit in SQLite.” [Online]. Available: https://www.sqlite.org/atomiccommit.html

[14] National Institute of Standards and Technology, Secure Hash Standard (SHS), FIPS PUB 180-4, 2015.

[15] FormDev Software, “FlatLaf — Flat Look and Feel.” [Online]. Available: https://www.formdev.com/flatlaf/

# APPENDIX

## A.1 Full source code:

https://github.com/MarcussRico/shelfwise-library-management

The repository contains the application (src/main/java/shelfwise), the JUnit tests (src/test/java), the catalogue data, the experiment runner, the raw results (results/results.json), the scripts that drew every figure, and a build script that compiles, tests and runs everything.

## A.2 Complete weekly PBL log and mentor sign-offs:

The full week-by-week log is given in Table 3.1 (Chapter 3); it spans the twelve-week PBL cycle beginning at the Zeroth Review on 27 June 2026.

## A.3 Self and Peer Assessment

| Team Member | Self-Rated Contribution (%) | Peer-Rated Contribution (%) | Remarks |
|---|---|---|---|
| Sai Charan A | 60% | 60% | Led the project; class design, service layer and JDBC, loan policies, atomic issue, audit ledger, simulator and experiments |
| Ashfaq Muhammed A | 40% | 40% | Swing interface, typo-tolerant search, catalogue loader with ISBN repair, JUnit suite and screenshots |

## A.4 PBL Poster

